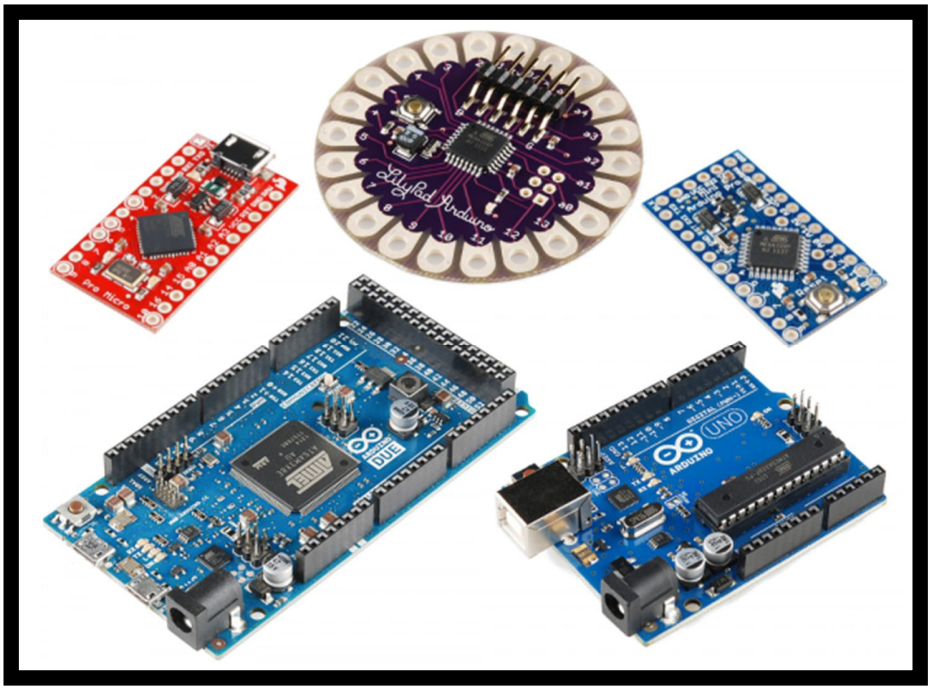
Hands – on workshop on Digital Systems & Applications with Microprocessors and Microcontrollers

[under the aegis of DBT STAR College scheme]

20 – 27 October 2021



organized by

Department of Physics and Internal Quality Assurance Cell

Acharya Narendra Dev College

(University of Delhi)

Govindpuri, Kalkaji, Delhi – 110 019, INDIA

Hands – on workshop on
Digital Systems & Applications with Microprocessors and Microcontrollers

organized by

Department of Physics and Internal Quality Assurance Cell (IQAC)

Acharya Narendra Dev College

[under the aegis of DBT STAR College scheme]

20 – 27 October 2021

Patron & STAR coordinator for College

Professor Ravi Toteja
Officiating Principal

Physics Department STAR coordinator

Professor Arijit Chowdhuri

Conveners

Dr. V. Bhasker Raj
Dr. Sanjay Kumar

Venue and Registration

Mr. Anil Kishore Godiyal

Hands – on workshop on
Digital Systems & Applications with Microprocessors and Microcontrollers

organized by

Department of Physics and Internal Quality Assurance Cell (IQAC)

Acharya Narendra Dev College

[under the aegis of DBT STAR College scheme]

20 – 27 October 2021

Programme

20 October 2021 (Wednesday)		
0930 – 1000	Registration	
1000 – 1015	Introduction	
1015 – 1030	Basics of Digital Electronics	Prof. Arijit Chowdhuri
1030 – 1045	How to get started	
1045 – 1100	Tea Break	
1100 – 1300	Hands - on session 1	
1300 – 1345	LUNCH Break	
1345 – 1500	Hands - on session 2	
1500 – 1515	Discussion on technical issues	

21 October 2021 (Thursday)		
0930 – 1030	Basics of Microprocessor	Dr. V. Bhasker Raj
1030 – 1130	Hands - on session 3	
1130 – 1145	Tea Break	
1145 – 1300	Hands - on session 4	
1300 – 1400	LUNCH Break	
1400 – 1500	Hands - on session 5	

1500 – 1530	Discussion and feedback session	

22 October 2021 (Friday)		
0930 – 1030	Applications of Microprocessor	Dr. V. Bhasker Raj
1030 – 1130	Hands - on session 6	
1130 – 1145	Tea Break	
1145 – 1300	Hands - on session 7	
1300 – 1400	LUNCH Break	
1400 – 1500	Hands - on session 8	
1500 – 1530	Discussion and feedback session	

23 October 2021 (Saturday)		
0930 – 1030	Concepts of Arduino Micro-controller	Mr. Jatinder Pal Singh
1030 – 1130	Hands - on session 9	
1130 – 1145	Tea Break	
1145 – 1300	Hands - on session 10	
1300 – 1400	LUNCH Break	
1400 – 1500	Hands - on session 11	
1500 – 1530	Discussion and feedback session	

25 October 2021 (Monday)		
0930 – 1030	Basics of Arduino Microcon-troller	Mr. Jatinder Pal Singh
1030 – 1130	Hands - on session 12	
1130 – 1145	Tea Break	
1145 – 1300	Hands - on session 13	
1300 – 1400	LUNCH Break	
1400 – 1500	Hands - on session 14	
1500 – 1530	Discussion and feedback session	

26 October 2021 (Tuesday)		
0930 – 1030	Applications of Arduino Mi-crocontroller	Mr. Jatinder Pal Singh

1030 – 1130	Hands - on session 15
1130 – 1145	Tea Break
1145 – 1300	Hands - on session 16
1300 – 1400	LUNCH Break
1400 – 1500	Hands - on session 17
1500 – 1530	Discussion and feedback session

27 October 2021 (Wednesday)		
0930 – 1030	Applications of Digital Electronics	Dr. V. Bhasker Raj
1030 – 1130	Discussion and feedback session	
1130 – 1230	Certificate distribution and vote of thanks	

Contents

1. Introduction 1
2. Need for Arduino 2
3. Types of Boards and Interface 3 - 4
4. IDE set up 5 - 9
5. Structure of an Arduino Program 10
6. Arduino Programming: Basics 11 -13
7. Projects
   - To blink an LED for a particular time period 14 - 15
   - To blink multiple LEDs 16 - 17
   - To obtain text on Serial Monitor for real time output 18
   - To detect change in input signal state 19 - 21
   - To convert analog input voltage and obtain it on a serial monitor 22
   - To obtain analog input voltage and plot it using Serial Plotter 23 - 24
   - To display the ASCII Table for different bytes available till date 25 - 26
   - To analyse a character entered by a user and give information about it 27-29
   - To use Arduino as a stopwatch 30 - 36
   - To vary the intensity of the LED using Arduino 37 - 38
   - To change decimal input to binary 39 - 41
   - To produce the Knightrider effect through LED display 42 - 44
   - To use Arduino as a function generator 45 - 46
   - To use Arduino for controlling two way traffic lights 47 - 51

- To measure the capacitance of a charging/discharging capacitor 52 - 54
- To use Arduino as a Voltmeter (0-30 V) 55 - 56
- To use Arduino as an Ammeter (0-3 mA) 57 - 58
- To use Arduino for data acquisition 59 - 61

8. Resources 62

INTRODUCTION

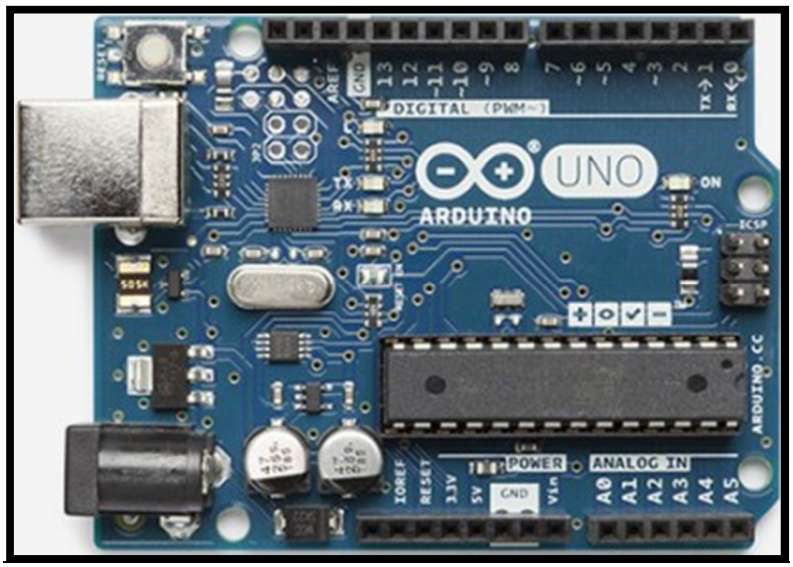
I am sure most of you in school must have seen the "Line Follower" robot and other simple robotics. But, when putting one's head about its functionality often ends up feeling baffled like a toddler. Not all of us are engineers nor do we belong from the technical background. Fortunately in the 21st century, things are getting really easy.

Introducing to you, state-of-THE-art technology, arduino.

WHAT IS ARDUINO?

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It is intended for everyone (artists, designers and hobbyists) making innovative projects. Arduino boards are able to read inputs as simple as light on a sensor, a finger on a button, temperature and then turn it into readable output. It is basically a microcontroller. A micro-controller can be comparable to a little stand alone computer. It is an extremely powerful device, which is able of executing a series of pre-programmed tasks and interacting with extra hardware devices for input/output, unlike a microprocessor which only processes and also needs externally connected hardware for its functioning. Arduino interacts with the physical world by taking inputs, processing (digital and analog signals) and then giving output in the usable format.

Thus, Arduino's main focus is to provide interface for programs to implement in real time.



One of the basic Arduino boards: UNO

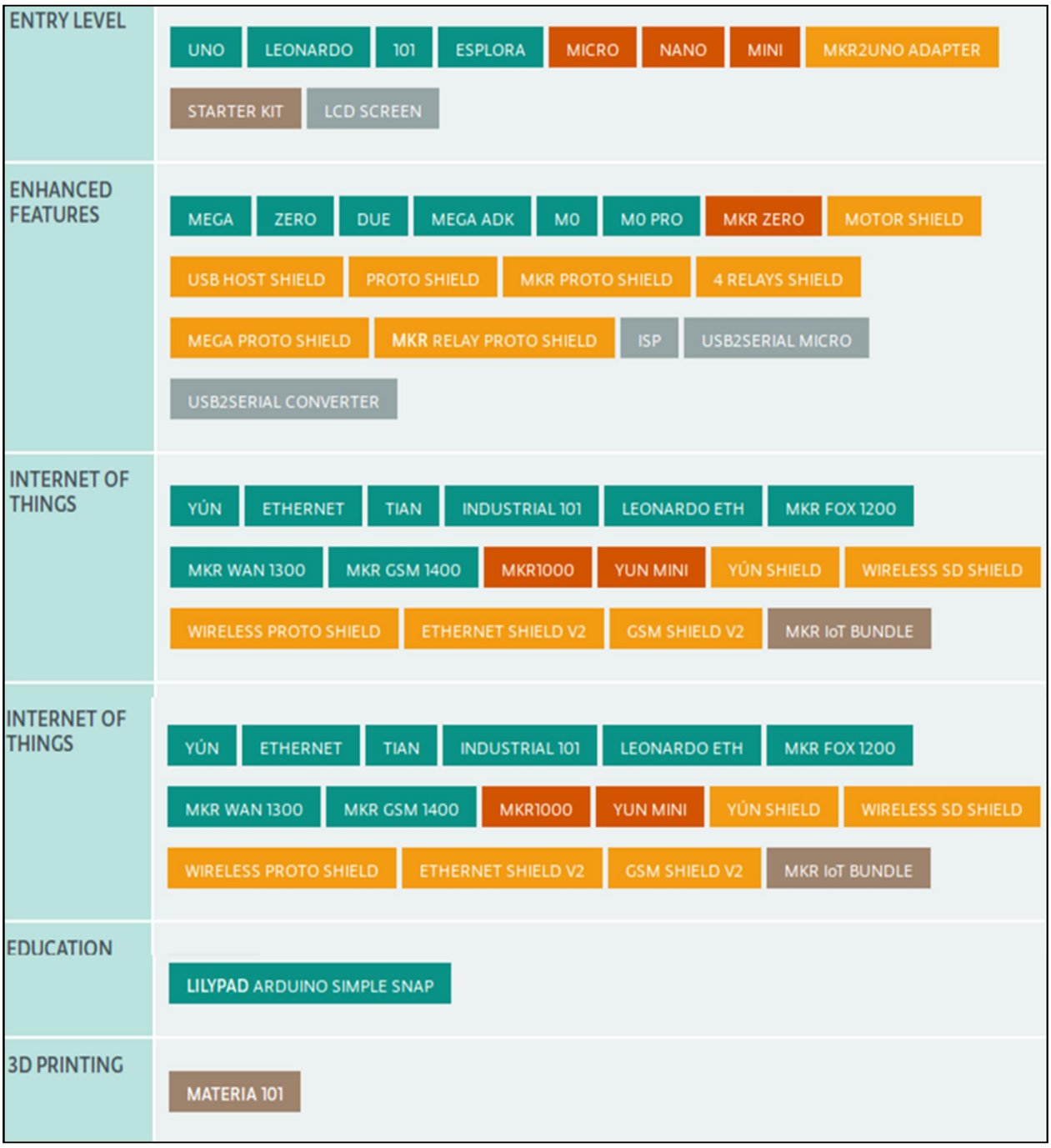
NEED FOR ARDUINO

Why ARDUINO ?

- **Open source and extensible software** - The Arduino software is published as open source tool, available for extension by experienced programmers. The language can be expanded through C++ libraries.
- **Open source and extensible hardware** - The plans of the Arduino boards are published under a Creative Commons license, so experienced circuit designers can make their own version of the module, extending it and improving it. Even beginners can build the breadboard version of the module.
- **Inexpensive** - Arduino boards are relatively inexpensive compared to other microcontroller platforms.
- **Cross-platform**- The Arduino Software (IDE) runs on Windows, Macintosh OSX, and Linux operating systems.
- **Simple, clear programming environment** - The Arduino Software (IDE) is easy-to-use for beginners, yet flexible enough for advanced users to take advantage of as well.
- **A stage to begin –** It gives a platform to implement the ideas without much problem even when it comes to the programming part.

TYPES OF ARDUINO BOARDS & INTERFACE

There are various types of Arduino boards available in the market. The basic differences between them are the number of inputs available to the user, physical appearance (rectangular/circular), followed by difference in performance through processor and storage. As the Arduino reaches the next level, it gets introduced to more features such as Ethernet connectivity or using external shields.



- The ‘Entry Level’ ones are the most basic microprocessors. The processor is available, one just has to feed in the program and view the working.
- The second level, ‘Enhanced Features’ has additional features besides the basic processing. For example, shield. Shield into your own Arduino for convenient breadboard prototyping and servo ports, with no soldering required.
- The third, ‘Internet of Things’, provides Arduino with another attribute of the Ethernet port attached to the board.
- ‘Education’, CTC 101, is a package for starters with a prototype to be made.
- In the ‘Wearable’ section, the Arduino is the size of a button being so small it can be “stitched” like a button. The working is as fine as any microcontroller with the size of a pea.
- Finally, the “3-D Printer’ is self explanatory. This Arduino board is used for 3 D printers.

Interactive Development Environment (IDE) Setup

Getting started

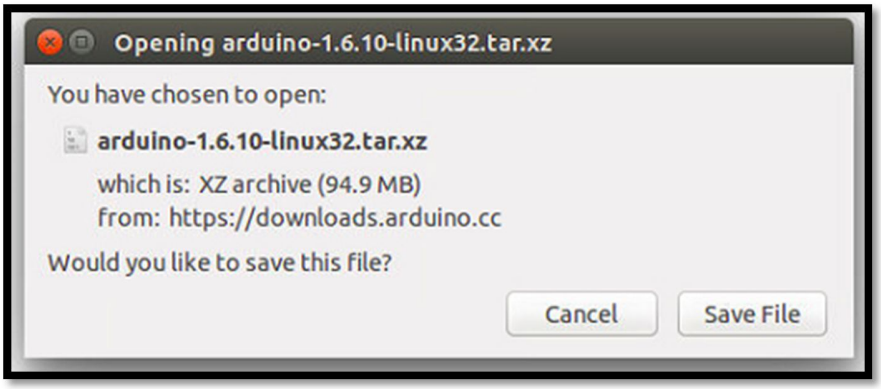
Any Arduino board can be programmed using the Arduino software (IDE). Using this Interactive Development Environment (IDE), Arduino can be programmed with any computer. You can either access this IDE on Arduino's official website and code online with the Arduino web editor or you can download the IDE on your computer and use it offline. To run this software, computer should have one of the following operating systems installed:

- Mac OS X or higher
- Windows XP 32 or 64-bit, or higher
- Linux 32 or 64-bit (Ubuntu or similar)

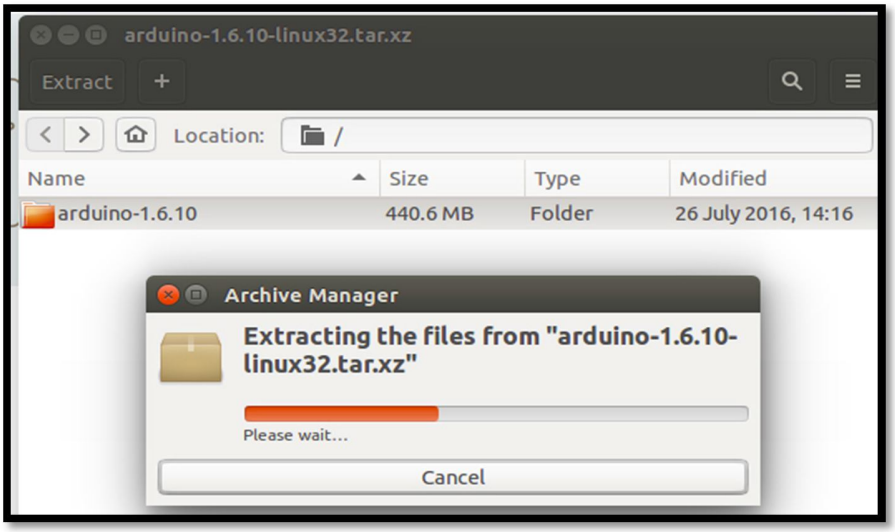
Let's download and install the Arduino IDE. According to the operating system follow the given instructions.

UBUNTU LINUX 16.04 AND LATER

1. Using a web browser such as Firefox, visit the software download page at https://www.arduino.cc/en/Main/Software
2. Click the Linux 32-bit or 64-bit link, depending on your system and start the download.



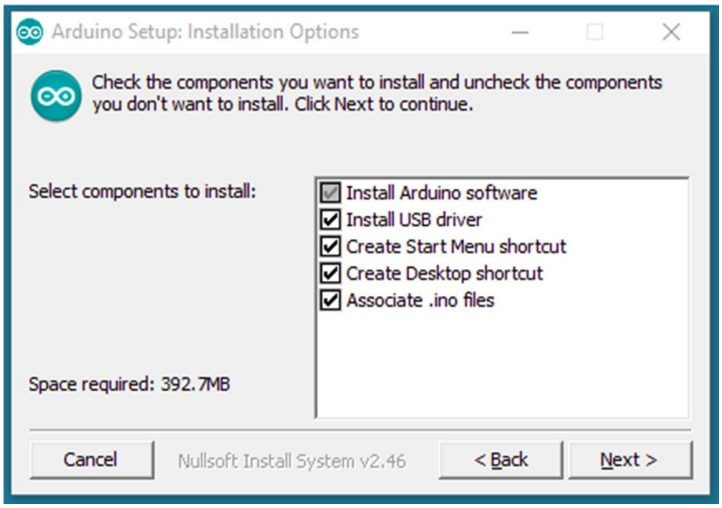
3. Extract the downloaded file to a suitable location. This would create a folder with the same name as that of the file.



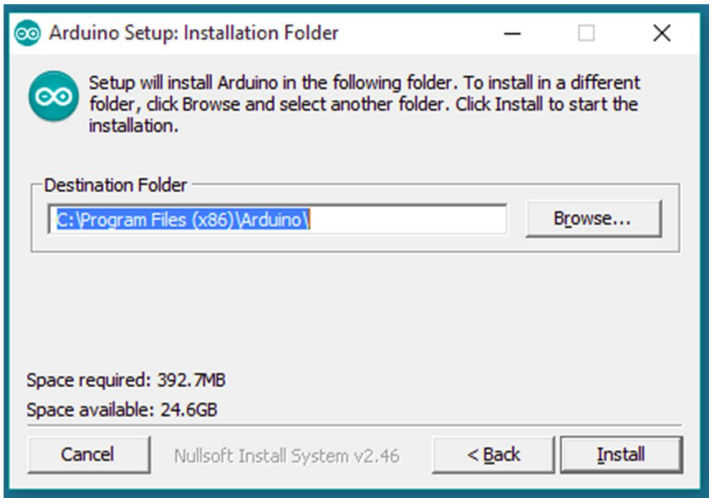
4. Open terminal, change the present working directory to the folder that was created in the last step.
5. Run the command chmod +x install.sh and then ./install.sh .

WINDOWS 7 AND LATER

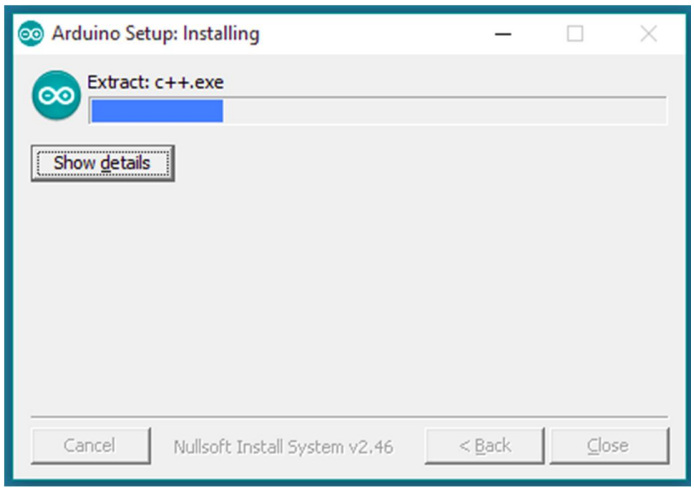
1. Using a web browser such as Firefox, visit the software download page at https://www.arduino.cc/en/Main/Software
2. Choose the version you wish to download. You can choose between the Installer (.exe) and the Zip packages. The first one installs directly everything you need to use the Arduino Software (IDE), including the drivers. With the Zip package you need to install the drivers manually. The Zip file is also useful if you want to create a portable installation.
3. When the download finishes, proceed with the installation and please allow the driver installation process when you get a warning from the operating system.



4. Choose the components to install



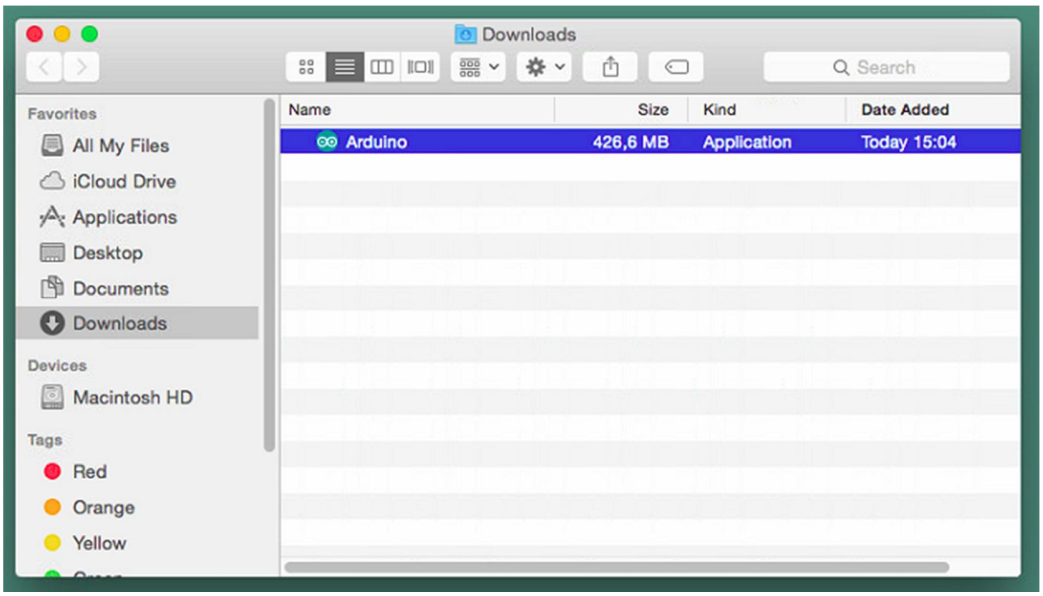
5. Choose the installation directory



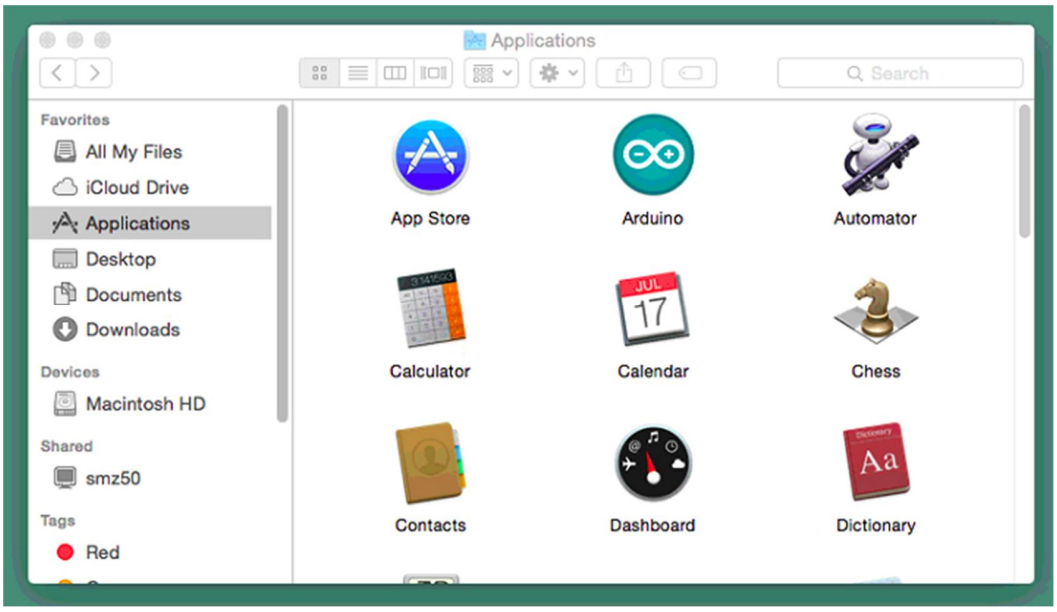
MAC OS X

1. Using a web browser, visit the software download page at

   https://www.arduino.cc/en/Main/Software

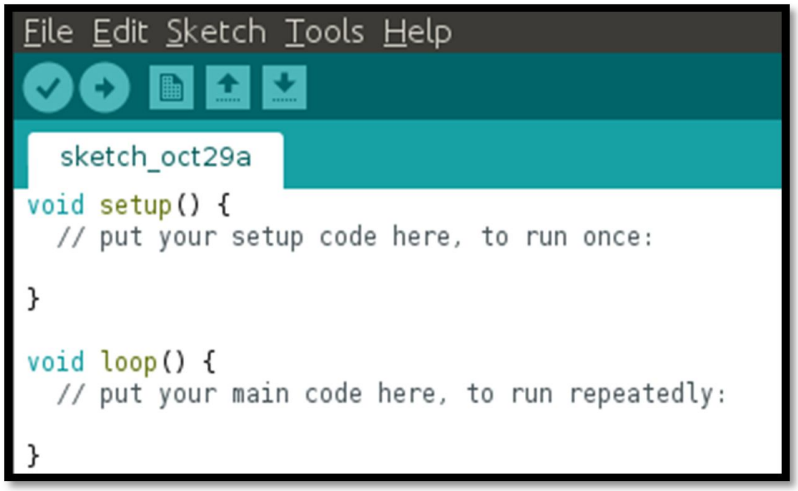


2. Download the file. The file is in Zip format; if you use Safari it will be automatically expanded. If you use a different browser you may need to extract it manually.
3. Copy the Arduino application into the Applications folder.



Make sure that you also have the matching USB cable for your arduino so that you are able to connect it to the computer.

After installing the Arduino IDE, when you run it the following window will open up:



Interactive Development Environment

Now that the IDE is running, we need to tell it which type of port the Arduino is connected to. Select Tools→Serial Port and select the suitable port.

Next, tell the IDE which Arduino you have connected. This is crucial, as Arduino boards do differ. For example, if you have the Uno, select Tools→Board→Arduino Uno.

Now your hardware and software are ready to work for you. All you need to do now is to write a code and upload it to your board.

STRUCTURE OF AN ARDUINO PROGRAM

Arduino is programmed with a C/C++ 'dialect', its syntax, statements and most of the functions are exactly like C++. It is really just C++ with domain specific libraries, all the C++ libraries cannot be used due to the limited memory and processing power on the board. All arduino programs can be divided in three main parts: structure, values (variables and constants), and functions. The arduino programming language is really great for beginners but have some limitations (like you must have all your files in the same folder).

A basic Arduino program(also known as a sketch) consists of two *functions* called **setup()** and **loop()** as explained below:

setup(): This function is called once when a sketch starts after power-up or reset. It is used to initialize variables, input and output pin modes, and other libraries needed in the sketch.

loop(): After *setup()* has been called, function *loop() i*s executed repeatedly in the main program. It controls the board until the board is powered off or is reset.

Apart from these two functions, we can introduce variables, constants, user-defined functions or inbuilt functions anywhere as per our requirement like any other ordinary program.

ARDUINO PROGRAMMING: BASICS

An Arduino sketch is a set of instructions created in order to accomplish a particular task. Now, every programming language has some rule regarding the way these instructions are written and executed. Arduino follows the same basic rules and syntax (the way the statements or instructions are written in the sketch) as that of C++. Let's take a look at some basic programming fundamentals which every programmer should know before writing a sketch:

- **Variables and Constants**

  A 'variable' as the name suggests is a part of the program whose value can be altered or changed during the program execution while a constant's value do not change during the program execution. Just like C++ the variables and constants can have data types: int, char, float, double, long etc.

- **Arrays**

  An 'array' is a set of values stored under a common name in a continuous memory location and can be accessed with an index number.

- **If...else statement**

  'If' tests whether a certain condition has been reached and executes certain statements. If the condition is not met, some other set of instructions are executed.

```
if (condition)
{
statements
}

else
{
statements
}
```

- **Loops**

  'For', 'while' and 'do while' statements are used to repeat a certain block of statements for a specified number of times.

```
For ( initialization; condition; updation)
{
statements
}
```

  The two other variants of looping are the while loop and the do while loop which can also be used as per the program's requirement.

```
while (condition)          do
{                          {
  statements                 statements
}                          }while (condition)
```

- **Functions**

  A function is a block of statements with a specific name that are executed whenever it is called. Function can be defined by the user or inbuilt like the mathematical functions sin(), cos(), sqrt(), pow() etc.

```
type FunctionName (parameters)
{
statements
}
```

- **Semicolon**

  A semicolon (;) musr be used to end a statement or else it will result in a compilation error.

  Int x=2428;

- **Comments**

  These are the non-executable statements in the program. Comments can be single statement like

```
// a single line comment
```

or a block comment like

```
/* This is a
block comment */
```

These were the C++/C basics, apart from these the operators and their precedence are the same as that in C++/C, but the Arduino programming language also has some terminologies and inbuilt functions of its own which are not generally a part of C++, some of them are:

- **pinMode(pin, mode)**
  It is used in the setup() block to configure a specified pin to behave as an INPUT or OUTPUT.
- **digitalRead(pin)**
  It is used to read the value from a specified pin. The result is either HIGH or LOW.
  For example, the statement is such as:
  value = digitalRead(pin)
- **digitalWrite(pin, value)**
  It is used to output logic level either HIGH or LOW to the specified pin.
- **analogRead(pin)**

It reads the value from a specified analog pin and stores the value in form of an integer ranging from 0 to 1023.

For example, the statement is such as:

value= analogRead(pin)

- **analogWrite(pin, value)**

It writes an analog value to the output pin. The value can be specified ranging from 0 to 255, 0 meaning 0 V and 255 generating a steady 5 V output at the specified pin.

- ***delay(ms)***

Pauses the program for the specified amount of time in milliseconds.

- **millis()**

It returns the time since the board began running the current program in milliseconds.

- **Serial.begin(rate)**

It opens the serial port and sets the baud rate for serial data transmission.

- **Serial.println(data)**

It prints data to the serial port.

PROJECTS

PROJECT 1

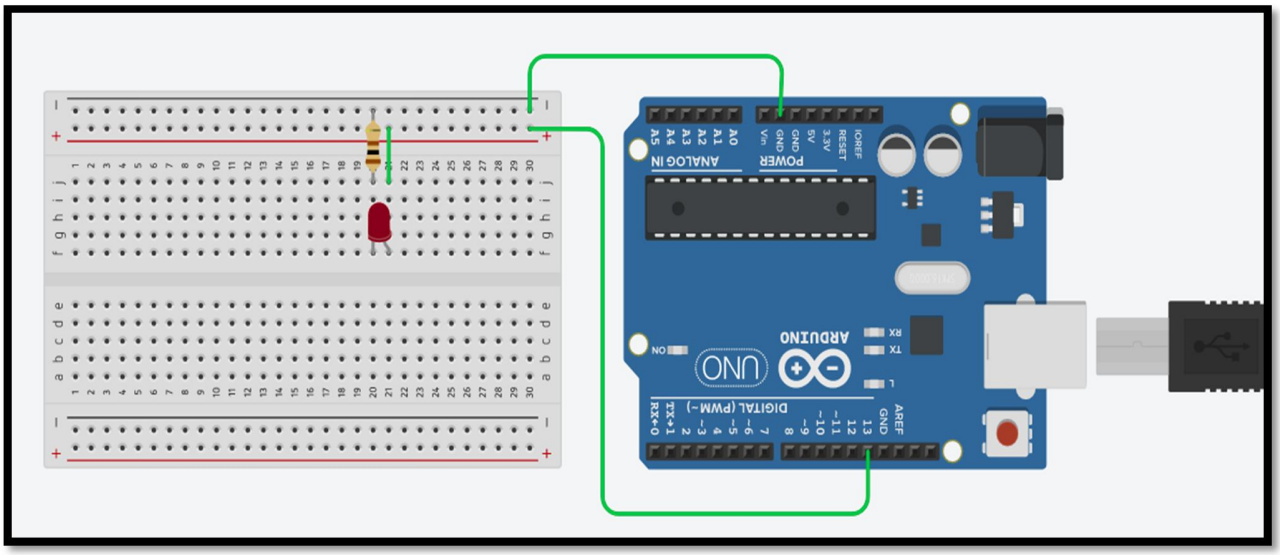
Objective: To blink an LED for a particular time period (say one second)

Hardware requirement:

To create this project, you will need:

- An LED
- One 100 ohm resistor
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*
Program to blink an LED

*/


int LED= 13; //sets up Output pin as PIN number 13


void setup()
{

pinMode(LED, OUTPUT);
}


void loop()
{
digitalWrite(LED, HIGH);     // turn the LED on (HIGH is the voltage level)

delay(1000);                 // wait for a second

digitalWrite(LED, LOW);    // turn the LED off by making the voltage LOW

delay(1000);               // wait for a second
}
```

PROJECT 2

Objective:

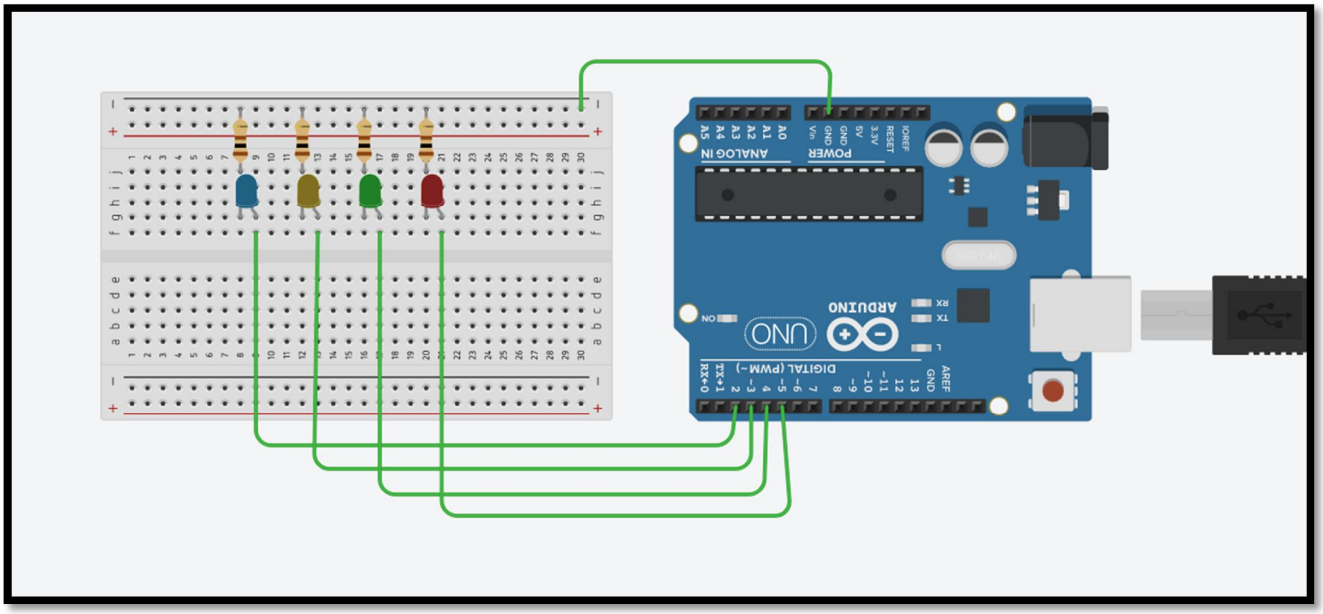
To blink multiple LEDs (say for 5 seconds).

Hardware requirement:

To create this project, you will need:

- Four LEDs
- Four 100 ohm resistors
- One breadboard
- Connecting wires
- Arduino and USB cable

The Circuit:



The Sketch:

```
/*
Program to blink multiple LEDs
*/

const int pin0=2;
const int pin1=3;
```

```
const int pin2=4;

const int pin3=5;

void setup()

{

pinMode(pin0, OUTPUT);     // initialize digital pins as an output.

pinMode(pin1, OUTPUT);

pinMode(pin2, OUTPUT);

pinMode(pin3, OUTPUT);

}

void loop()

{

digitalWrite(pin0, HIGH);     // turn the LED on (HIGH is the voltage level)

delay(5000);                                    // waits for a 5 second

digitalWrite(pin0, LOW);                        // turn the LED off by making the voltage LOW

digitalWrite(pin1, HIGH);                       // turn the LED on (HIGH is the voltage level)

delay(5000);                                    // waits for a 5 second

digitalWrite(pin1, LOW);                        // turn the LED off by making the voltage LOW

digitalWrite(pin2, HIGH);                       // turn the LED on (HIGH is the voltage level)

delay(5000);                                    // waits for a 5 second

digitalWrite(pin2, LOW);                        // turn the LED off by making the voltage LOW

digitalWrite(pin3, HIGH);                       // turn the LED on (HIGH is the voltage level)

delay(5000);                                    // waits for a 5 second

digitalWrite(pin3, LOW);                        // turn the LED off by making the voltage LOW

}
```

PROJECT 3

Objective:

To obtain text on Serial Monitor for real time output.

Note: Serial Monitor tool gives the ability to visualize your data in real time. Serial Monitor is obtained by ***Tools>Serial Monitor*** *in the main menu.*

Hardware requirement:

To create this project, you will need:

- Arduino
- USB cable

(No external circuitry is required for this project)

The Sketch:

```
/*
Program to use  Digital Read Serial
(Tools > Serial Plotter menu).
*/

void setup()

{

Serial.begin(9600);  //Sets the Baud Rate of the USB Port to 9600 PWM signals

}

void loop()

{
Serial.println("Hello. Welcome to the Next Gen hardware-software interface: ARDUINO");

delay(2000);        // delay for 2 seconds

Serial.println("This text is printed on Serial Monitor Plotter");

delay(2000);        // delay for 2 seconds
}
```

PROJECT 4

Objective:

To detect change in input signal state (Turns on the LED every four button pushes by checking the ***modulo*** *of the button push counter).*

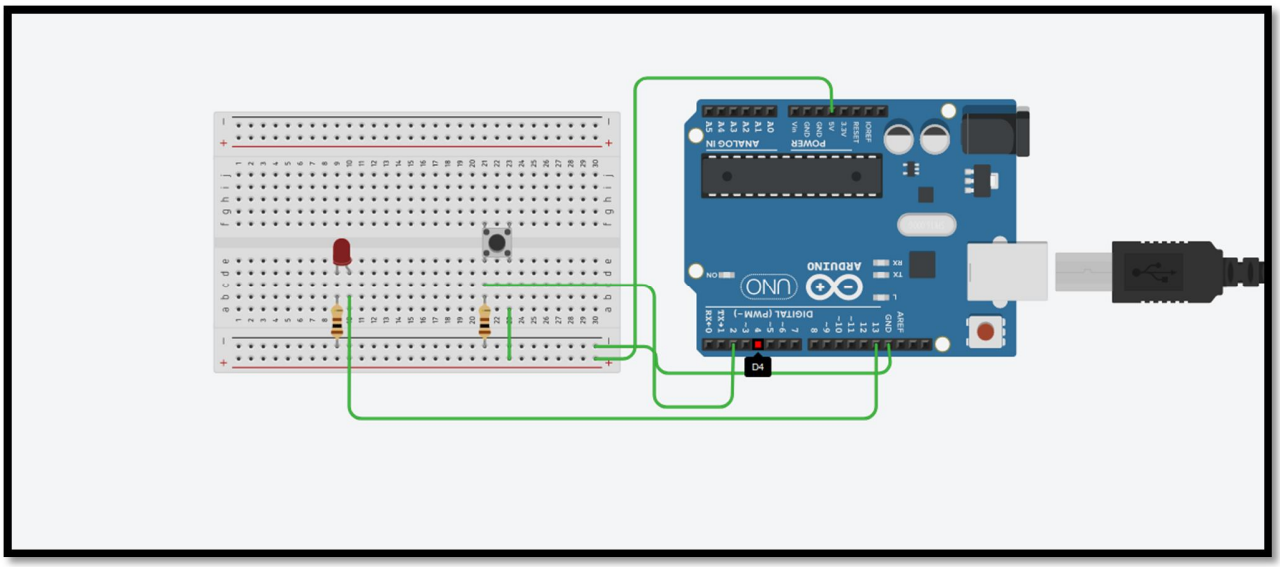
Note:The modulo function gives you the remainder of the division of two numbers. The basis of the modulo is ***if-else*** *condition.*

Hardware requirement:

To create this project, you will need:

- Push button
- One LED
- Two100 ohm resistors
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*

Program for Sate of Change of Detection

*/


// this constant won't change:

const int  buttonPin = 2;    // the pin that the pushbutton is attached to
const int ledPin = 13;       // the pin that the LED is attached to

// Variables will change:

int buttonPushCounter = 0;          // counter for the number of button presses

int buttonState = 0;                // current state of the button

int lastButtonState = 0;            // previous state of the button

void setup()
{

pinMode(buttonPin, INPUT);     // initialize the button pin as a input:


pinMode(ledPin, OUTPUT);      // initialize the LED as an output:


Serial.begin(9600);                // initialize serial communication:

}

void loop()

{

buttonState = digitalRead(buttonPin);     // read the pushbutton input pin:


if (buttonState != lastButtonState)          // compare the buttonState to its previous state

{
```

```
if (buttonState == HIGH)                // if the state has changed, increment the counter
{

buttonPushCounter++;                    // if the current state is HIGH then the button

Serial.println("On");

Serial.print("number of button pushes:  ");

Serial.println(buttonPushCounter);

}

else                                    // if the current state is LOW then the button
{
Serial.println("Off");
}

delay(50);                              // Delay a little bit to avoid bouncing

}

// save the current state as the last state

//for next time through the loop

lastButtonState = buttonState;

/* Turns on the LED every four button pushes by
checking the modulo of the button push counter.
The modulo function gives you the remainder of
the division of two numbers:
*/

if (buttonPushCounter % 4 == 0)

{
digitalWrite(ledPin, HIGH);
}

else

{
digitalWrite(ledPin, LOW);
}
```

}

PROJECT 5

Objective:

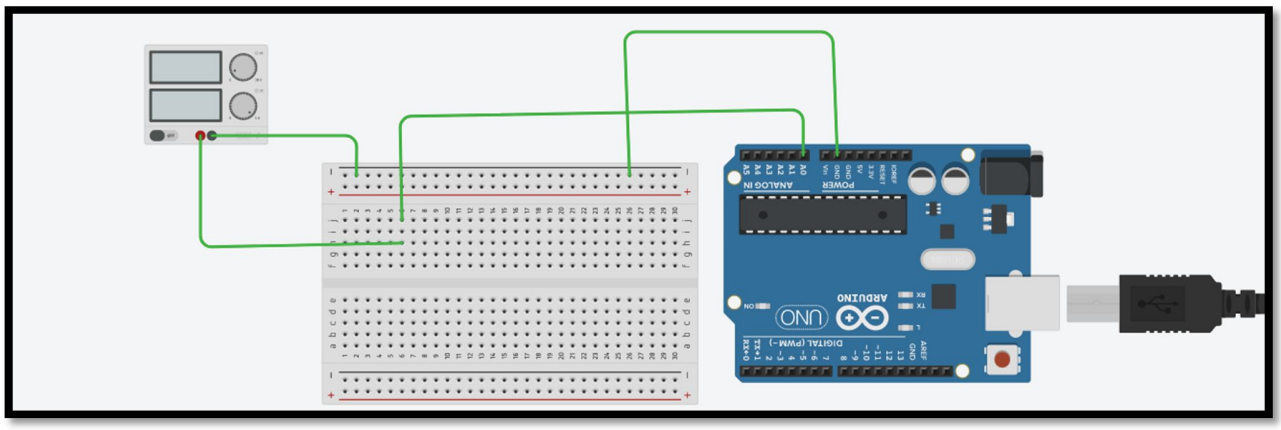
To convert analog input voltage and obtain it on a Serial Monitor. The analog reading (which goes from 0 - 1023) is therefore converts to a voltage ranging from 0 – 5V.

Hardware requirement:

To create this project, you will need:

- Analog input
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*
Program to Read Analog Voltage
Reads an analog input on pin 0, converts it to voltage, and prints the result to the Serial Monitor.
*/
void setup()
{ Serial.begin(9600);                // initialize serial communication at 9600 bits per second:
}
void loop()
{
int sensorValue = analogRead(A0);            // read the input on analog pin 0:
float voltage = sensorValue * (5.0 / 1023.0);   // Convert the analog reading (which goes from 0 -
                                                //  1023) to a voltage (0 - 5V):
```

```
Serial.println(voltage);                    // print out the value you read:
}
```

PROJECT 6

Objective:

To obtain analog input voltage and plot it using Serial Plotter.

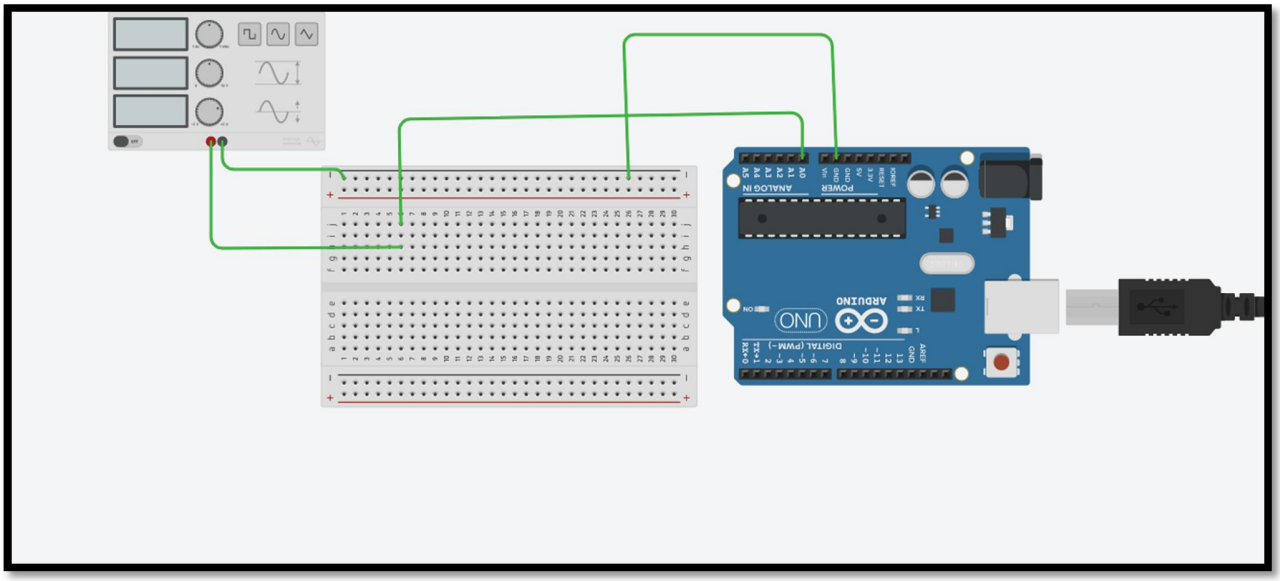
Note: The Serial plotter is an offline tool to visualize data and troubleshoot the code offline. Serial Plotter is obtained by ***Tools>Serial Plotter*** *in the main menu.*

Hardware requirement:

To create this project, you will need:

- An analog input source
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*
```

Program to plot Analog Voltage using the Serial plotter
Tools>Serial Plotter

```
*/

void setup()

{

Serial.begin(9600);     // initialize serial communication at 9600 bits per second:
}


void loop()

{

int sensorValue = analogRead(A0);                  // read the input on analog pin 0:


float voltage = sensorValue * (5.0 / 1023.0);   // Converts the analog reading (which goes from 0 –
                                                 // 1023) to a voltage (0 – 5V):


Serial.println(voltage);                         // print out the value you read:

}
```

PROJECT 7

Objective:

To display the ASCII(American Standard Code for Information Interchange) Table for different bytes available till date (Program shows Decimal, Hexadecimal, Octal, Binary information of each byte).

Hardware requirement:

To create this project, you will need:

- Arduino
- USB cable

(No external circuitry is required for this project)

The Sketch:

```
/*
ASCII table

Prints out byte values in all possible formats:
- as raw binary values
- as ASCII-encoded decimal, hex, octal, and binary values

*/

void setup()

{

Serial.begin(9600);  //Initialize serial and wait for port to open:


Serial.println("ASCII Table ~ Character Map");

}


int thisByte = 33;       // first visible ASCIIcharacter '!' is number 33:
// you can also write ASCII characters in single quotes.
// for example, '!' is the same as 33, so you could also use this:
// int thisByte = '!';
```

```
void loop()

{
// prints value unaltered, i.e. the raw binary version of the byte.
// The Serial Monitor interprets all bytes as ASCII, so 33, the first number,
// will show up as '!'


Serial.write(thisByte);

Serial.print(", dec: ");


Serial.print(thisByte);              // prints value as string as an ASCII-encoded decimal (base 10).

Serial.print(", hex: ");

Serial.print(thisByte, HEX);     // prints value as string in hexadecimal (base 16):

Serial.print(", oct: ");

Serial.print(thisByte, OCT);    // prints value as string in octal (base 8);

Serial.print(", bin: ") ;            // prints value as string in binary (base 2) also prints ending line break:

Serial.println(thisByte, BIN);


if (thisByte == 126)              // you could also use if (thisByte == '~')

{

while (true)

{
continue;
}

}

thisByte++;              // go on to the next character (This is called Updation in technical terms)
}

// The last printed last visible character '~' or 126 and then the program stops.
```

PROJECT 8

Objective:

To analyse a character entered by a user and give information about it.

Hardware requirement:

To create this project, you will need:

- Arduino
- USB cable

(No external circuitry is required for this project)

The Sketch:

```
/*
Program for Character Analysis
*/
void setup()
{

Serial.begin(9600);              // Open serial communications and wait for port to open/

Serial.println("Send any byte and I'll tell you everything I can about it");

Serial.println();
}
void loop()
{

if (Serial.available() > 0)              // get any incoming bytes:

{
int thisChar = Serial.read();

Serial.print("You sent me: \'");     // tell the user what was sent:

Serial.write(thisChar);
```

```
Serial.print("\'  ASCII Value: ");

Serial.println(thisChar);


if (isAlphaNumeric(thisChar))           // analyze what was sent:
{

Serial.println("it's alphanumeric");

}

if (isAlpha(thisChar))
{

Serial.println("it's alphabetic");

}

if (isAscii(thisChar))

{
Serial.println("it's ASCII");

}

if (isWhitespace(thisChar))

{

Serial.println("it's whitespace");
}
if (isControl(thisChar))

{

Serial.println("it's a control character");
}
if (isDigit(thisChar))

{
Serial.println("it's a numeric digit");
}
if (isGraph(thisChar))
{
Serial.println("It's a printable character that's not whitespace");
```

```
}
if (isLowerCase(thisChar))
{

Serial.println("it's lower case");

}

if (isPrintable(thisChar))

{
Serial.println("It's printable");

}
if (isPunct(thisChar))

{
Serial.println("It's punctuation");

}

if (isSpace(thisChar))

{

Serial.println("It's a space character");

}

if (isUpperCase(thisChar))
{

Serial.println("It's upper case");

}

if (isHexadecimalDigit(thisChar))
{
Serial.println("it's a valid hexadecimaldigit (i.e. 0 - 9, a - F, or A – F)");

}

Serial.println();        // add some space and ask for another byte:

Serial.println("Give me another byte:");

Serial.println();
```

```
}
}
```

PROJECT 9

Objective:

To use Arduino as a Stopwatch.

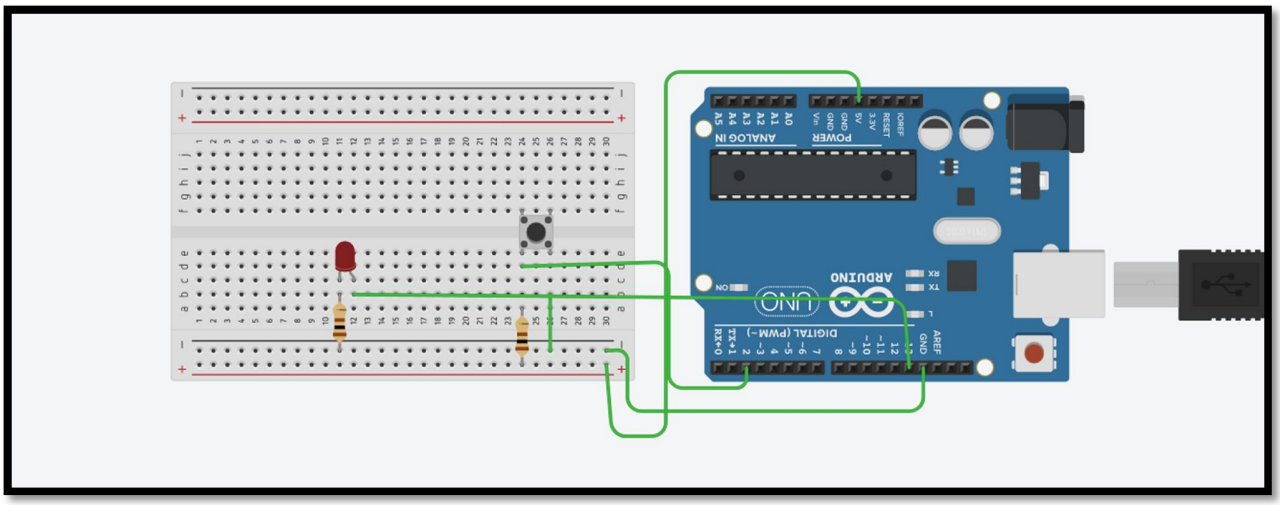
Note: A pushbutton is used to start and stop the timer. The program uses ***millis()*** *function to obtain a minutest value when the Arduino works as a Stopwatch.*

Hardware requirement:

To create this project, you will need:

- Push Button
- Arduino
- USB cable
- One LED
- Two 100 ohm resistors

The Circuit:



The Sketch:

```
/*

This is an example for a Stopwatch

*/


int ledPin = 13;                // LED connected to digital pin 13

int buttonPin = 2;              // button on pin 2

int value = LOW;                // previous value of the LED

int buttonState;                // variable to store button state

int lastButtonState;            // variable to store last button state

int blinking;                   // condition for blinking - timer is timing

int frameRate = 100;            // the frame rate (frames per second) at which the stopwatch runs -
                                //Change to suit

long interval = (1000/frameRate);   // blink interval

long previousMillis = 0;        // variable to store last time LED was updated

long startTime ;                // start time for stop watch

long elapsedTime ;              // elapsed time for stop watch

int fractional;                 // variable used to store fractional part of Frames

int fractionalSecs;             // variable used to store fractional part of Seconds

int fractionalMins;             // variable used to store fractional part of Minutes

int elapsedFrames;              // elapsed frames for stop watch

int elapsedSeconds;             // elapsed seconds for stop watch

int elapsedMinutes;             // elapsed Minutes for stop watch

char buf[10];                   // string buffer for itoa function


void setup()
```

```
{

Serial.begin(9600);

pinMode(ledPin, OUTPUT);        // sets the digital pin as output

pinMode(buttonPin, INPUT);      // not really necessary, pins default to INPUT anyway

digitalWrite(buttonPin, HIGH);  // turn on pullup resistors. Wire button so that press shorts pin to
                                // ground.

}


void loop()

{

digitalWrite(ledPin, LOW);              // Initiate LED and Step Pin States


buttonState = digitalRead(buttonPin);   // Check for button press, read the button state and store


if (buttonState == LOW && lastButtonState == HIGH  &&  blinking == false)
/* check for a high to low transition if true then found a new button press while clock is not running -
start the clock */

{

startTime = millis();                                   // store the start time

blinking = true;                                        // turn on blinking while timing

delay(10);                                              // short delay to debounce switch

lastButtonState = buttonState;                  // store buttonState in lastButtonState, to compare next time

}



else if (buttonState == LOW && lastButtonState == HIGH && blinking == true)
/* check for a high to low transition if true then found a new button press while clock is running - stop
the clock and report */
```

```
{
blinking = false;                                   // turn off blinking, all done timing

lastButtonState = buttonState;                // store buttonState in lastButtonState, to compare next time


// Routine to report elapsed time

elapsedTime =   millis() - startTime;              // store elapsed time

elapsedMinutes = (elapsedTime / 60000L);

elapsedSeconds = (elapsedTime / 1000L);             // divide by 1000 to convert to seconds - then cast to
                                                    // an int to print

elapsedFrames = (elapsedTime / interval);           // divide by 100 to convert to 1/100 of a second - then
                                                    // cast to an int to print

fractional = (int)(elapsedFrames % frameRate);      // use modulo operator to get fractional part of 100
                                                    // Seconds

fractionalSecs = (int)(elapsedSeconds % 60L);       // use modulo operator to get fractional part of 60
                                                    // Seconds

fractionalMins = (int)(elapsedMinutes % 60L);       // use modulo operator to get fractional part of 60
                                                    // Minutes



if (fractionalMins < 10)

{
// pad in leading zeros
Serial.print("0");                            // add a zero

}


Serial.print(itoa(fractionalMins, buf, 10));       // convert the int to a string and print it

Serial.print(":");                          //print a colon.


if (fractionalSecs < 10)

{                                           // pad in leading zeros
Serial.print("0");                         // add a zero
```

```
}

Serial.print(itoa(fractionalSecs, buf, 10));        // convert the int to a string and print it

Serial.print(":");                                  //print a colon.

if (fractional < 10)

{                                          // pad in leading zeros

Serial.print("0");                                // add a zero

}

Serial.print(itoa(fractional, buf, 10));            // convert the int to a string and print it

}

else

{
lastButtonState = buttonState;                      // store buttonState in lastButtonState, to compare next time

}

/*

run commands at the specified time interval
blink routine - blink the LED while timing
check to see if it's time to blink the LED; that is, the difference
between the current time and last time we blinked the LED is larger than
the interval at which we want to blink the LED.

*/

if ( (millis() - previousMillis > interval) )

{

if (blinking == true)
```

```
{
previousMillis = millis();                  // remember the last time we blinked the LED


digitalWrite(ledPin, HIGH);                 // Pulse the LED for Visual Feedback


elapsedTime =   millis() - startTime;           // store elapsed time

elapsedMinutes = (elapsedTime / 60000L);      // divide by 60000 to convert to minutes - then cast to an
                                              // int to print

elapsedSeconds = (elapsedTime / 1000L);       // divide by 1000 to convert to seconds - then cast to an
                                              // int to print

elapsedFrames = (elapsedTime / interval);     // divide by 40 to convert to 1/25 of a second - then cast to
                                              // an int to print

fractional = (int)(elapsedFrames % frameRate);  // use modulo operator to get fractional part of 25
                                                // Frames

fractionalSecs = (int)(elapsedSeconds % 60L);  // use modulo operator to get fractional part of 60
                                               // Seconds

fractionalMins = (int)(elapsedMinutes % 60L);   // use modulo operator to get fractional part of 60
                                                // Minutes


if (fractionalMins < 10)
{                                   // pad in leading zeros

Serial.print("0");                        // add a zero

}


Serial.print(itoa(fractionalMins, buf, 10));   // convert the int to a string and print it

Serial.print(":");                          //print a colon.


if (fractionalSecs < 10)
{                                   // pad in leading zeros
Serial.print("0");                        // add a zero

}
```

```
Serial.print(itoa(fractionalSecs, buf, 10));   // convert the int to a string and print it

Serial.print(":");                          //print a colon.


if (fractional < 10)
{                                       // pad in leading zeros
Serial.print("0");                          // add a zero

}

Serial.print(itoa((fractional), buf, 10));  // convert the int to a string and print it

Serial.println();
}


else

{

digitalWrite(ledPin, LOW);                // turn off LED when not blinking

}
}


}
```

PROJECT 10

Objective:

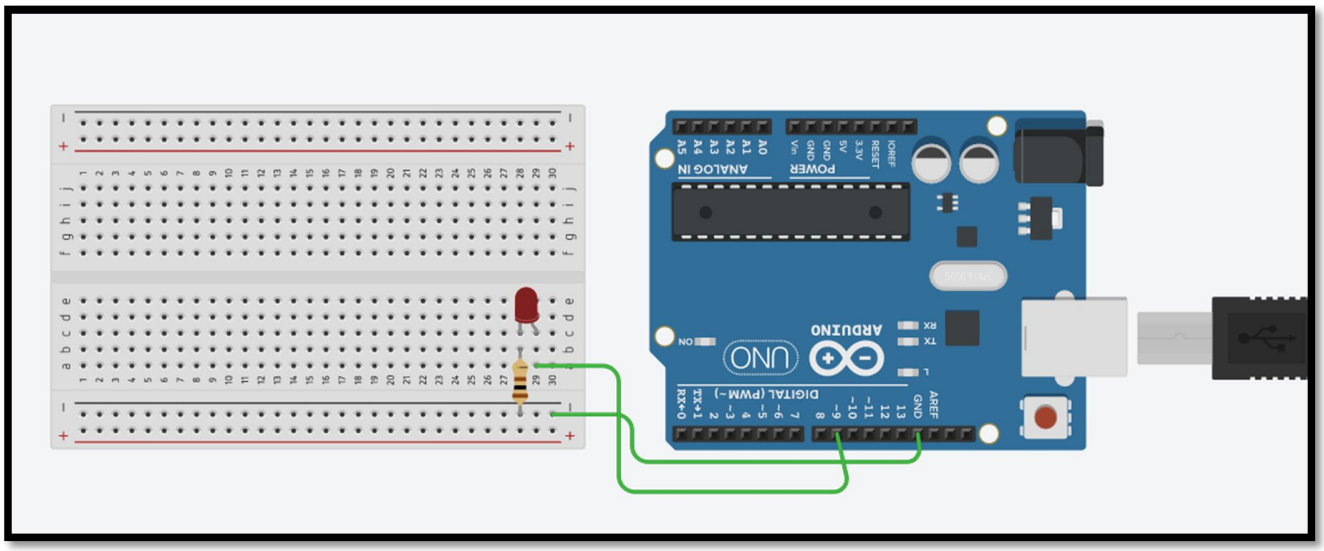
To vary the intensity of the LED using Arduino.

Hardware requirement:

To create this project, you will need:

- One LED
- One100 ohm resistor
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

/*

```
Program to vary the intensity of an LED

*/

int led = 9;                    // the PWM pin the LED is attached to

int brightness = 0;             // how bright the LED is

int fadeAmount = 5;             // how many points to fade the LED by


void setup()
{

pinMode(led, OUTPUT);           // declare pin 9 to be an output:

}


void loop()

{

analogWrite(led, brightness);   // set the brightness of pin 9:


brightness = brightness + fadeAmount;   // change the brightness for next time through the loop:


if (brightness <= 0 || brightness >= 255)   // reverse the direction of the fading at the ends of the fade:


{
fadeAmount = -fadeAmount;

}

delay(30);                      // wait for 30 milliseconds to see the dimming effect

}
```

PROJECT 11

Objective:

To change Decimal input to Binary.

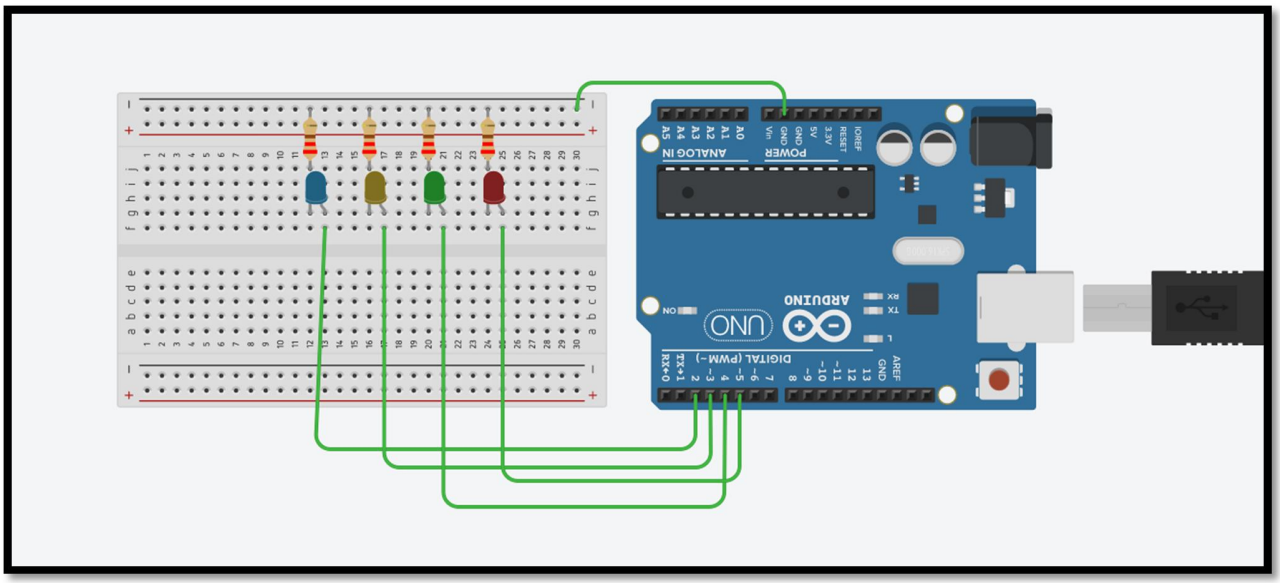
Note: A special in-built function ***Serial.parseInt()*** *is used to get the value as it is without any change. The underlying concept is to convert the decimal number (up to 16 as the output being used is 4 LEDs) to binary ($2^n_{n=0,1,2..}$). It can convert up to 16 decimal values (0-15).*

Hardware requirement:

To create this project, you will need:

- Four LEDs
- Four220 ohm resistors
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*
Decimal to Binary
*/

int outputLED[]={5,4,3,2};
int serialInput;
void setup()
{
Serial.begin(9600);
// Open serial communications and wait for port to open:
pinMode(outputLED[0], OUTPUT);
pinMode(outputLED[1], OUTPUT);
pinMode(outputLED[2], OUTPUT);
pinMode(outputLED[3], OUTPUT);
while (!Serial)
{
; // wait for serial port to connect. serialInputeeded for native USB port only
}
Serial.println("Enter the number to be converted to binary(0-15) ");
Serial.println();
}
void loop()
{
if (Serial.available() > 0) // digitalWrite(outputLED[0],HIGH);
{
serialInput = Serial.parseInt();
//Initialzes serialInput as the first valid integer from the serial monitor
Serial.println(serialInput);
if(serialInput>=8)
{
serialInput=serialInput-8;
digitalWrite(outputLED[0],HIGH);
Serial.println(serialInput);
}
else

digitalWrite(outputLED[0],LOW);
if(serialInput>=4)
{
serialInput=serialInput-4;
Serial.println(serialInput);
digitalWrite(outputLED[1],HIGH);
}
else
digitalWrite(outputLED[1], LOW);
if(serialInput>=2)
```

```
{
serialInput=serialInput-2;
Serial.println(serialInput);
digitalWrite(outputLED[2],HIGH);
}
else
digitalWrite(outputLED[2],LOW);
if(serialInput>=1)
{
serialInput=serialInput-1;
Serial.println(serialInput);
digitalWrite(outputLED[3],HIGH);
}
else
digitalWrite(outputLED[3],LOW);
}
}
```

PROJECT 12

Objective:

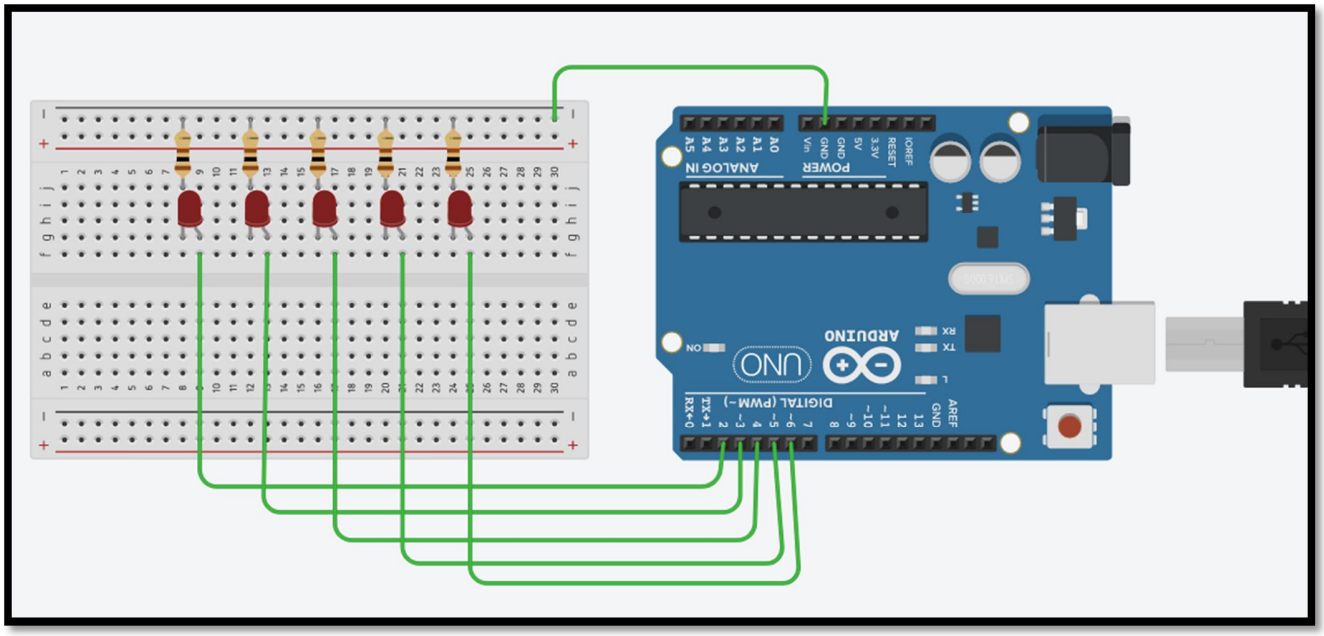
To produce the Knightrider effect through LED display. (5 LEDs are used to emulate the effect and produce a wave-like pattern.

Hardware requirement:

To create this project, you will need:

- Five LEDs
- Five 100 ohm resistors
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

/*

```
Program to display the Knightrider effect

*/

void setup()

{
pinMode(2,OUTPUT);          // LED 1 control pin is set up as an output

pinMode(3,OUTPUT);          // same for LED 2 to LED 5

pinMode(4,OUTPUT);

pinMode(5,OUTPUT);

pinMode(6,OUTPUT);

}

void loop()

{

digitalWrite(2, HIGH);          // Turn LED 1 ON

delay(500);                     // wait half a second

digitalWrite(2, LOW);          // Turn LED 1 off


digitalWrite(3, HIGH);        // and repeat for LED 2 to 5

delay(500);

digitalWrite(3, LOW);

digitalWrite(4, HIGH);

delay(500);

digitalWrite(4, LOW);

digitalWrite(5, HIGH);

delay(500);
```

```
digitalWrite(5, LOW);

digitalWrite(6, HIGH);

delay(500);

digitalWrite(6, LOW);

digitalWrite(5, HIGH);

delay(500);

digitalWrite(5, LOW);

digitalWrite(4, HIGH);

delay(500);

digitalWrite(4, LOW);
digitalWrite(3, HIGH);

delay(500);

digitalWrite(3, LOW);

}
```

PROJECT 13

Objective:

To use Arduino as a function generator.

Hardware requirement:

- Arduino
- USB cable

(No external circuitry is required for this project)

The Sketch:

```
/*
  Arduino used as Function Generator

  Tools>Serial Plotter
*/

void setup()
{
Serial.begin(9600);
}

void loop()

{

  float angle=0;

  for(angle=0;angle<=90;angle=angle+0.1)  //Sine Wave & Cosine Wave

  {
    float sina=sin(angle);

    float cosa=cos(angle);

Serial.print(sina);

Serial.print(" ");
```

```
Serial.println(cosa);

delay(10);
  }
for(float i=0; i<=90; i=i+1)  //Sawtooth Wave
  {

Serial.println(i);

delay(10);

  }

for(int a=0; a<=100; a++)  //Square Wave

  {
int b=0;

Serial.println(b);

delay(10);
  }
for(int a=0; a<=100; a++)

  {
int b=100;
Serial.println(b);
delay(10);

  }

for(float i=0;i<=100;i=i+1)   //Triangular wave

  {

Serial.println(i);

delay(10);

  }

for(float i=100;i>=0;i=i-1)

  {
Serial.println(i);
delay(10);
```

```
  }

}
```

PROJECT 14

Objective:

To use Arduino for controlling two-way traffic lights.

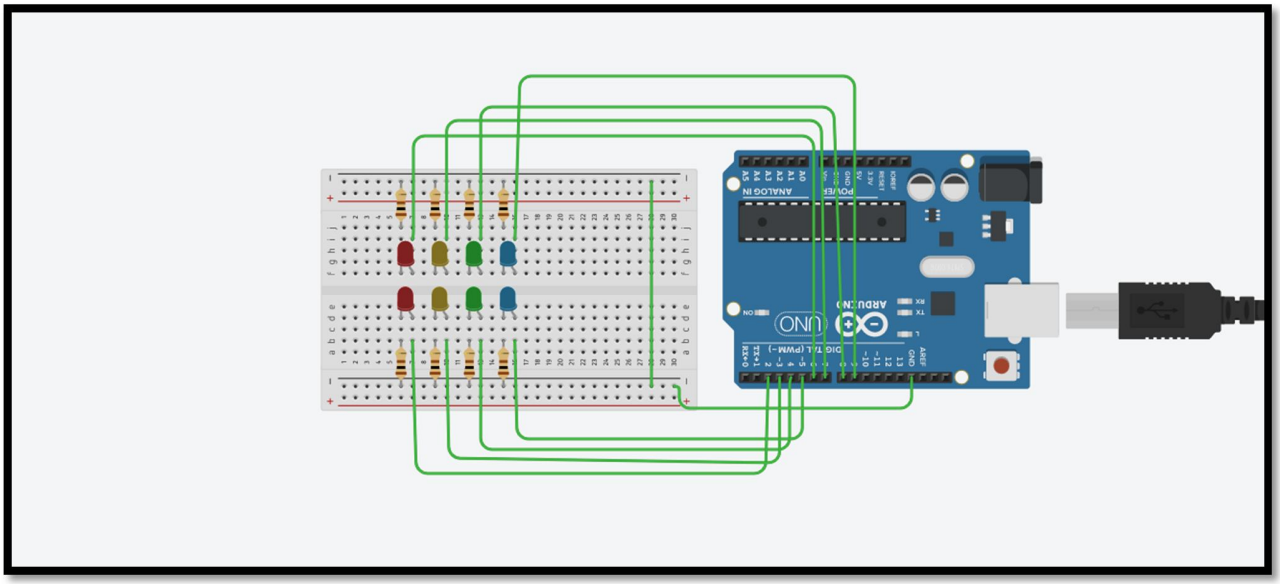
Note: One light shouldn't cross the other to ensures safe walking for pedestrians.

Hardware requirement:

To create this project, you will need:

- Two red LEDs
- Two yellow LEDs
- Two green LEDs
- Two blue LEDs
- Eight 100 ohm resistors
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*

Program for a traffic light system

*/

int trafficLights1[] = {2,3,4,5};                    // red, yellow, green, pedestrians led pins

int trafficLights2[] = {6,7,8,9};                    // red, yellow, green, pedestrians led pins

int situations = 4;

int duration[] = {8000,3000,10000,3000};    // duration of each situation

long previousCars = 0;

long previousPeds = 0;

long interval = 300;                                //blink interval for pedestrians

int ledState = LOW;

int state;
```

```
int i = 0;

void setup()

{

for(int i = 0; i < 4; i++)

{

        pinMode(trafficLights1[i], OUTPUT);

        pinMode(trafficLights2[i], OUTPUT);
}

      Serial.begin(9600);
}

void loop()

{
      unsigned long currentMillis = millis();

      if(currentMillis - previousCars < duration[i])
{
            situation(i);
}

else

{
            previousCars = currentMillis;

            if(i >= situations)
{
                  i = 0;

}

else

{
                  i++;
            }
      }
}
```

```
void activateTrafficLight1(String lights, int pedestrians)

{
        for(int x = 0; x < 3; x++)
        {
                if(lights[x] == '0') state = LOW;

                if(lights[x] == '1') state = HIGH;

                digitalWrite(trafficLights1[x], state);

        }

        if(pedestrians == 1)
{
                blinkPed(trafficLights1[3]);
}

else

{
                digitalWrite(trafficLights1[3], LOW);
}

}

void activateTrafficLight2(String lights, int pedestrians)

{
        for(int x = 0; x < 3; x++)

        {
                if(lights[x] == '0') state = LOW;

                if(lights[x] == '1') state = HIGH;

                digitalWrite(trafficLights2[x], state);
        }

        if(pedestrians == 1)
{

blinkPed(trafficLights2[3]);

        }
else
```

```
{
        digitalWrite(trafficLights2[3], LOW);
}

}


void situation(int i)
{
    switch(i)

{
        case 0:
            activateTrafficLight1("100",1); // 100 means red ON, yellow OFF, green OFF

            activateTrafficLight2("001",0); // the second parameter is for pedestrians

            break;                                          // 1 is ON and 0 is OFF

        case 1:
            activateTrafficLight1("110",0); // 110: red ON, yellow ON, green OFF

            activateTrafficLight2("010",0);

            break;

        case 2:

            activateTrafficLight1("001",0);

            activateTrafficLight2("100",1);

            break;

        case 3:

            activateTrafficLight1("010",0);

            activateTrafficLight2("110",0);

            break;
    }
}


void blinkPed(int ped)
{
```

```
        unsigned long currentMillis = millis();

        if(currentMillis - previousPeds > interval)
{

                previousPeds = currentMillis;

                if (ledState == LOW)

                ledState = HIGH;

                else

                ledState = LOW;

                digitalWrite(ped, ledState);
        }
}
```

PROJECT 15

Objective Achieved:

To measure the capacitance of a charging/discharging capacitor (capacitance visualizer)

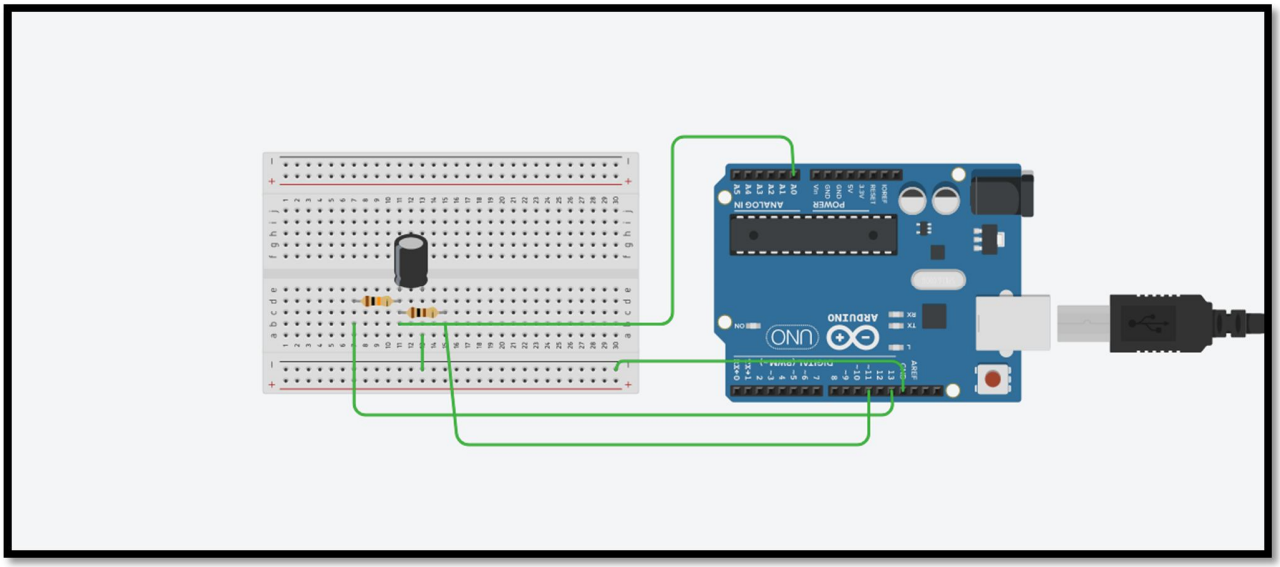
*Note: A capacitor will charge, through a resistor, with time constant, defined as T seconds where T = R * C. The particular number used in one of the loops* ***647*** *is* ***63.2%*** *of 1023, which corresponds to charging of the capacitor then discharging after reaching its full potential.*

Hardware requirement:

To create this project, you will need:

- A 10k ohm resistor
- A 100 ohm resistor
- A capacitor (unknown value)
- Bread board
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*  RCTiming_capacitance_meter
*
* Theory   A capcitor will charge, through a resistor, in one time constant, defined as T seconds where
*    TC = R * C
*
*    TC = time constant period in seconds
*    R = resistance in ohms
*
*/

//#define is used for defining constants

#define analogPin      0         // analog pin for measuring capacitor voltage
#define chargePin      13        // pin to charge the capacitor - connected to one end of the charging resis-
tor
#define dischargePin   11        // pin to discharge the capacitor
#define resistorValue  10000.0F   // change this to whatever resistor value you are using
// F formatter tells compliler it's a floating point value

unsigned long startTime;

unsigned long elapsedTime;

float microFarads;              // floating point variable to preserve precision, make calculations
```

```
float nanoFarads;

void setup()
{
pinMode(chargePin, OUTPUT);     // set chargePin to output

digitalWrite(chargePin, LOW);

Serial.begin(9600);             // initialize serial transmission for debugging
}

void loop()
{
digitalWrite(chargePin, HIGH);  // set chargePin HIGH and capacitor charging

startTime = millis();

while(analogRead(analogPin) < 648){
// 647 is 63.2% of 1023, which corresponds to full-scale voltage
}

elapsedTime= millis() - startTime;

// convert milliseconds to seconds ( 10^-3 ) and Farads to microFarads ( 10^6 ),  net 10^3 (1000)

microFarads = ((float)elapsedTime / resistorValue) * 1000;

Serial.print(elapsedTime);       // print the value to serial port

Serial.print(" mS    ");         // print units and carriage return

if (microFarads > 1)
{
Serial.print((long)microFarads);       // print the value to serial port

Serial.println(" microFarads");         // print units and carriage return

}

else

{
// if value is smaller than one microFarad, convert to nanoFarads (10^-9 Farad).

// This is  a workaround because Serial.print will not print floats
```

```
nanoFarads = microFarads * 1000.0;      // multiply by 1000 to convert to nanoFarads (10^-9 Farads)

Serial.print((long)nanoFarads);         // print the value to serial port

Serial.println(" nanoFarads");          // print units and carriage return
}

/* dicharge the capacitor  */

digitalWrite(chargePin, LOW);           // set charge pin to  LOW

pinMode(dischargePin, OUTPUT);          // set discharge pin to output

digitalWrite(dischargePin, LOW);        // set discharge pin LOW

while(analogRead(analogPin) > 0){       // wait until capacitor is completely discharged

}

pinMode(dischargePin, INPUT);           // set discharge pin back to input
}
```

PROJECT 16

Objective:

To use Arduino as a voltmeter (0-30 V).

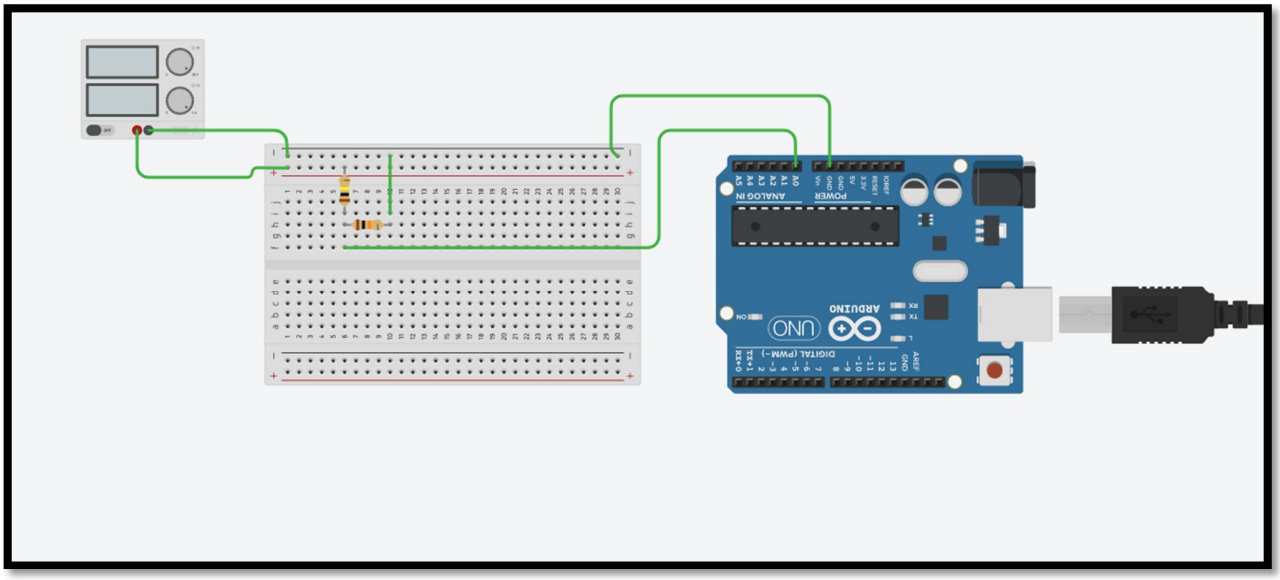
Note: The input is obtained in such a manner that one is able to get values of the voltage running across the resistors attached to the circuit.

Hardware requirement:

To create this project, you will need:

- A 100k ohm resistor
- A 10k ohm resistor
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*

Program to create a Digital Voltmeter using Arduino

*/
int analogInput = 1;

float Vout = 0.00;

float Vin = 0.00;

float R1 = 100000.00; // resistance of R1 (100K)

float R2 = 10000.00; // resistance of R2 (10K)

int val = 0;

void setup()
{
pinMode(analogInput, INPUT);      //assigning the input port

Serial.begin(9600);               //BaudRate
}
void loop()
{

val = analogRead(analogInput);        //reads the analog input
```

```
Vout = (val * 5.00) / 1024.00;      // formula for calculating voltage out i.e. V+, here 5.00

Vin = Vout / (R2/(R1+R2));        // formula for calculating voltage in i.e. GND

if (Vin<0.09)//condition

{

Vin=0.00;                //statement to quash undesired reading !

}

Serial.println("\t Voltage of the given source = ");

Serial.print(Vin);

delay(1000); //for maintaining the speed of the output in serial moniter

}
```

PROJECT 17

Objective:

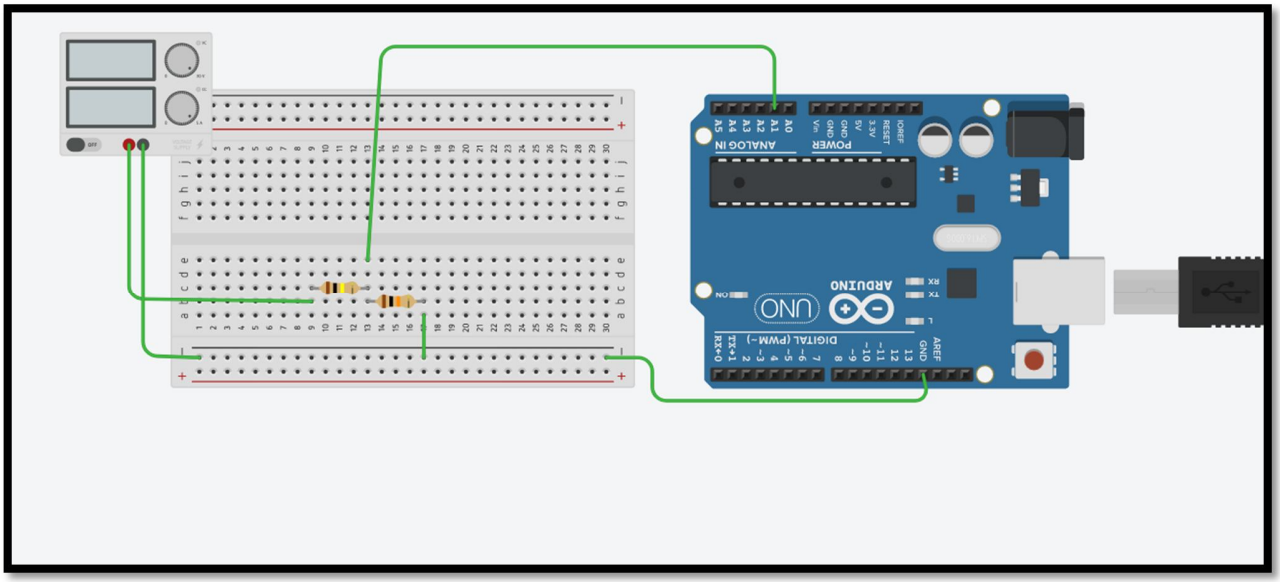
To use Arduino as an Ammeter (0-3 mA).

Hardware requirement:

To create this project, you will need:

- A 100k ohm resistor
- A 10k ohm resistor
- One breadboard
- Connecting wires
- Arduino
- USB cable

The Circuit:



The Sketch:

```
/*

   Program to create a Digital Ammeter using Arduino

*/
intanalogInput = 1;

float Vout = 0.00;

float Vin = 0.00;

double I = 0.00;

float R1 = 100000.00; // resistance of R1 (100K)

float R2 = 10000.00; // resistance of R2 (10K)

intval = 0;

void setup()
{
pinMode(analogInput, INPUT); //assigning the input port

Serial.begin(9600);        //BaudRate

}
void loop()
{
```

```
val = analogRead(analogInput);     //reads the analog input

Vout = (val * 5.00) / 1024.00;    // formula for calculating voltage out i.e. V+, here 5.00

   Vin = Vout / (R2/(R1+R2));       // formula for calculating voltage in i.e. GND

   if (Vin<0.09)//condition

   {
     Vin=0.00;                    //statement to quash undesired reading !
  }

    I=Vin*1000/(R1+R2);

Serial.println();

Serial.print("CURRENT(in mA)= ");

Serial.print(I);


delay(1000);                    //for maintaining the speed of the output in serial moniter

}
```

PROJECT 18

Objective:

To use Arduino for Data Acquisition.

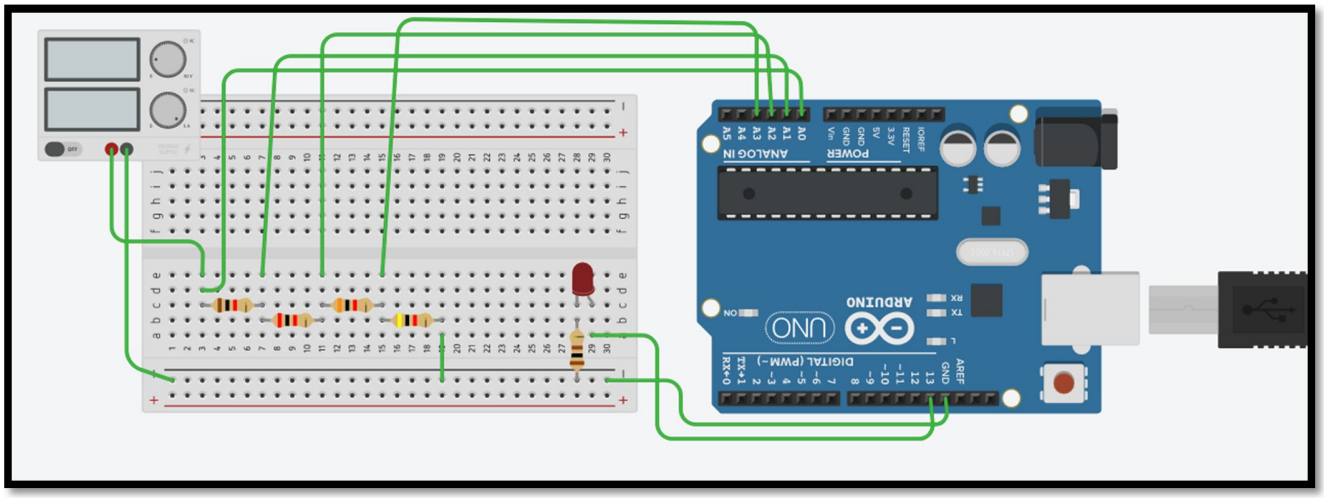
Hardware requirement:

To create this project, you will need:

- A 1k ohm resistor
- A 2k ohm resistors
- A 3k ohm resistor
- A 4k ohm resistors
- Voltage Source
- One breadboard
- Connecting wires
- Arduino

- USB cable

The Circuit:



The Sketch:

```
/*
  Program created for Data Acquisition

*/

int LED=13;                          //Sets pin 13 as LED output

constint analogpin0=A0;              //Sets pin 0 as input from analog 1

constint analogpin1=A1;              //Sets pin 1 as input from analog 2

constint analogpin2=A2;              //Sets pin 2 as input from analog 3

constint analogpin3=A3;              //Sets pin 3 as input from analog 4

float sensorvalue[4]={0,0,0,0};      // Array initialized

void setup()

{
Serial.begin(9600);

pinMode(LED,OUTPUT);
```

```
  }

void loop()
{
digitalWrite(LED,HIGH);
readsensors();                    //Function defined by the programmer(1)
sendValue();                      //Function defined by the programmer(2)
digitalWrite(LED,LOW);
}
void readsensors()                //Function defined by the programmer(1)
{
sensorvalue[0]=analogRead(analogpin0);
sensorvalue[1]=analogRead(analogpin1);
sensorvalue[2]=analogRead(analogpin2);
sensorvalue[3]=analogRead(analogpin3);
}
void sendValue()                  //Function defined by the programmer(2)
{
Serial.print('\n');
  for(inti=0;i<4;i++)
  {         float voltage = sensorvalue[i] * (5.0 / 1023.0);        //It changes the analogous values to 0-
255
Serial.print(i);
Serial.print("$ ");
Serial.print(voltage);
```

```
Serial.print('\n');            //String '/n' used for new line

                    delay(3000);              //A delay of 3 seconds
  }


Serial.print(' ');

}
```

RESOURCES

- **www.arduino.cc**
- **www.google.com**
- **www.google.com/images**
- **www.elprocus.com/difference-between-avr-arm-8051-and-pic-microcontroller/**
- **www.circuits.io**
- **https://circuitdigest.com**

This is just the beginning, with a tool as powerful as the Arduino, the possibilities are endless!